

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 8: BLUETOOTH DATA INTERFACING

SECTION 2

GROUP 17

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday, 1 December 2025

Date of Submission: Monday, 8 December 2025

NO.	GROUP MEMBERS	MATRIC NO.
1.	MUHAMMAD IMAN FARIHIN BIN MOHD IMRAN	2313813
2.	MUHAMMAD SYAZANI AKMAL BIN MOHD NASARUDIN	2314517
3.	AININ SOFIYA BINTI KAMARUZAMAN	2314916

ABSTRACT

This laboratory experiment focuses on the integration of a mechatronics system by developing a wireless temperature monitoring and control setup using Bluetooth technology. The system utilizes either an ESP32 development board or an Arduino paired with an HC-01 Bluetooth module to interface with a DHT22 temperature and humidity sensor. The microcontroller continuously acquires real-time temperature data and transmits it wirelessly to a paired computer or smartphone through a Bluetooth serial connection. A Python script or serial terminal on the remote device is used to receive and display the data, enabling effective remote monitoring. In addition to data transmission, the system supports bi-directional communication, allowing users to send simple control commands such as “FAN ON” and “FAN OFF” back to the microcontroller. These commands trigger predefined actions, demonstrating the concept of remote environmental control. The results confirm stable Bluetooth connectivity, reliable data acquisition, and accurate visualization of temperature changes over time. The control commands were also received and executed successfully, validating the closed-loop communication capability of the system. Overall, the experiment successfully achieved its objectives and illustrates fundamental principles of Bluetooth interfacing, sensor integration, wireless communication, and basic actuator control within a mechatronics framework.

TABLE OF CONTENTS

INTRODUCTION	4
MATERIALS AND EQUIPMENTS	4
EXPERIMENTAL SETUP	5
METHODOLOGY	6
DATA COLLECTION	13
DATA ANALYSIS	13
RESULTS	14
DISCUSSION	15
CONCLUSION	16
RECOMMENDATIONS	16
REFERENCES	17
APPENDICES	18
ACKNOWLEDGEMENTS	19

INTRODUCTION

Modern mechatronics systems frequently leverage wireless communication for remote monitoring and control of sensors and actuators. Bluetooth technology provides a low-cost, accessible method for short-range serial data transmission, allowing microcontrollers to communicate with external devices such as computers or smartphones. Bluetooth operates via a master-slave configuration, establishing a paired connection where data can be sent and received reliably over short distances.

The DHT22 sensor, used in this experiment, measures ambient temperature and humidity using a thermistor and capacitive humidity sensor, and outputs digital signals suitable for microcontroller interfacing. Microcontrollers such as the ESP32 or Arduino can read these digital signals, process the data, and transmit it wirelessly via Bluetooth. Additionally, microcontrollers can receive incoming commands from paired devices to control actuators, enabling a closed-loop system where monitoring and control occur simultaneously.

This experiment aims to develop a wireless temperature monitoring and control system using either an ESP32 or Arduino with an HC-01 Bluetooth module. The objectives are to accurately acquire real-time temperature and humidity data from the DHT22 sensor, transmit it to a remote device for display and logging, and implement bi-directional communication to respond to control commands such as "FAN ON" or "FAN OFF." It is hypothesized that the system will reliably collect and transmit data without significant delay or loss and correctly execute received commands, thereby demonstrating a functioning wireless monitoring and control system.

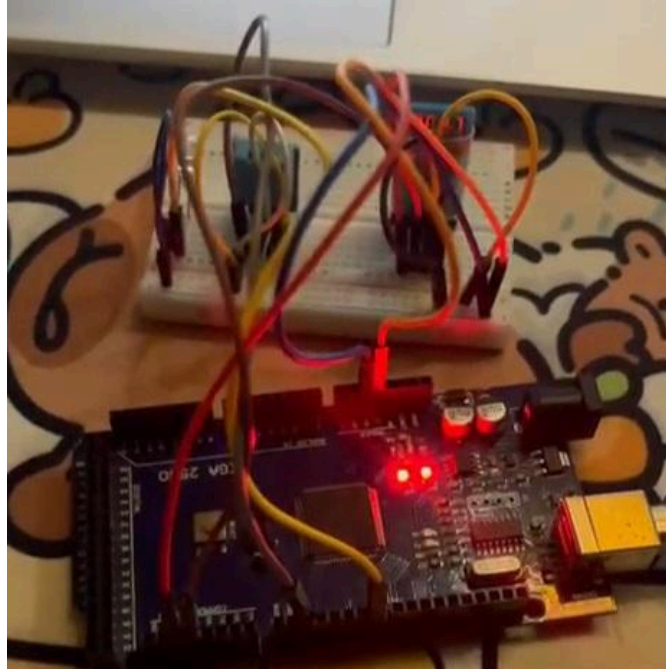
MATERIALS AND EQUIPMENTS

The materials and equipment that we used in this experiment are

- Arduino uno board
- Temperature sensor -DHT22
- Bluetooth module - HC-01
- Smartphone with Bluetooth support
- Breadboard
- Jumper wires

EXPERIMENTAL SETUP

1. The DHT22 temperature and humidity sensor was connected to the Arduino by wiring its VCC pin to the 5V pin, its GND pin to the GND pin, and its DATA pin to a selected digital input pin on the Arduino (such as digital pin 4, as used in the program).
2. The HC-01 Bluetooth module was connected to the Arduino by connecting its VCC pin to the 5V supply, GND to the Arduino GND, TXD (HC-01) to the Arduino's RX pin, and RXD (HC-01) to the Arduino's TX pin. When using the SoftwareSerial library, the TXD and RXD pins were connected to Arduino pins 2 and 3 as configured in the sketch.
3. Jumper wires were used to complete all electrical connections between the Arduino board, the DHT22 sensor, the HC-01 module, and the breadboard, ensuring a secure and organized setup.
4. The Arduino board was connected to the PC through a USB cable, supplying power to the board and enabling program uploading as well as serial debugging using the Arduino IDE.
5. Wireless communication in this experiment relied entirely on Bluetooth, as the Arduino does not support Wi-Fi. The HC-01 module functioned as the wireless interface for transmitting temperature data and receiving control commands from the paired device.



METHODOLOGY

1. The circuit was assembled as shown in Figure 1, using an Arduino UNO connected to the DHT22 temperature sensor and the HC-01 Bluetooth module through the appropriate power and signal pins.
2. An Arduino program was written and uploaded to the UNO to initialize the DHT22 sensor, read temperature data at regular intervals, and enable Bluetooth serial communication using the HC-01 module through the SoftwareSerial library.
3. The program output was verified by checking the Arduino Serial Monitor to confirm correct sensor readings and ensuring that the same temperature values were transmitted through Bluetooth.
4. A basic serial Bluetooth terminal application was installed on a smartphone to allow wireless communication with the Arduino system.
5. The smartphone was paired with the HC-01 Bluetooth module through the phone's Bluetooth settings menu.
6. The HC-01 module was connected via the terminal app, and incoming temperature readings were monitored to confirm successful wireless data transmission.
7. The command was sent from the smartphone through the Bluetooth app to test the Arduino's ability to receive and interpret control instructions.
8. Upon receiving the command, the Arduino processed it and simulated fan control by switching an LED on or off according to the programmed temperature threshold.
9. A Python script was developed to read temperature data from the Bluetooth serial port and plot a real-time temperature-versus-time graph.
10. Temperature readings were recorded over a period of operation, allowing observation of environmental changes and evaluation of the system's automated control response.

Coding used:

Arduino

```
#include <DHT.h>

#define DHTPIN 2

#define DHTTYPE DHT11 // or DHT22

DHT dht(DHTPIN, DHTTYPE);
```



```
int ledPin = 7;

void setup() {

    Serial.begin(9600);      // USB Serial Monitor

    Serial1.begin(9600);     // Bluetooth via Serial1

    dht.begin();

    pinMode(fanPin, OUTPUT);

    Serial.println("System Started...");

    Serial1.println("Bluetooth Connected...");

}

void loop() {

    float h = dht.readHumidity();
```

```
float t = dht.readTemperature();
```

```
if (!isnan(h) && !isnan(t)) {
```

```
    Serial.print("TEMP:");
```

```
    Serial.print(t);
```

```
    Serial.print(", HUM:");
```

```
    Serial.println(h);
```

```
    Serial1.print("TEMP:");
```

```
    Serial1.print(t);
```

```
    Serial1.print(", HUM:");
```

```
    Serial1.println(h);
```

```
}
```

```
if (Serial1.available()) {
```

```
String cmd = Serial1.readStringUntil('\n');

cmd.trim();


Serial.print("BT Command: ");

Serial.println(cmd);


if (cmd == "FAN_ON") {

    digitalWrite(ledPin, HIGH);

    Serial1.println("Fan ON");

}


if (cmd == "FAN_OFF") {

    digitalWrite(ledPin, LOW);

    Serial1.println("Fan OFF");

}

}
```

```
    delay(1000);  
  
}
```

Python

```
import serial  
  
import time  
  
# Change COM port to your Arduino port (e.g., COM3, COM4, etc.)  
  
PORT = "COM5"  
  
BAUD = 9600  
  
try:  
  
    ser = serial.Serial(PORT, BAUD, timeout=1)  
  
    time.sleep(2) # wait for Arduino reset  
  
    print(f"Connected to {PORT} at {BAUD} baud.\n")  
  
except:
```

```
print("Error: Could not open serial port.")

exit()

while True:

    if ser.in_waiting > 0:

        line = ser.readline().decode('utf-8').strip()

        # Example line: TEMP:28.5,HUM:65.2

        print("Raw:", line)

        if "TEMP" in line:

            try:

                # Split data

                parts = line.replace("TEMP:", "").split(",HUM:")

                temp = float(parts[0])

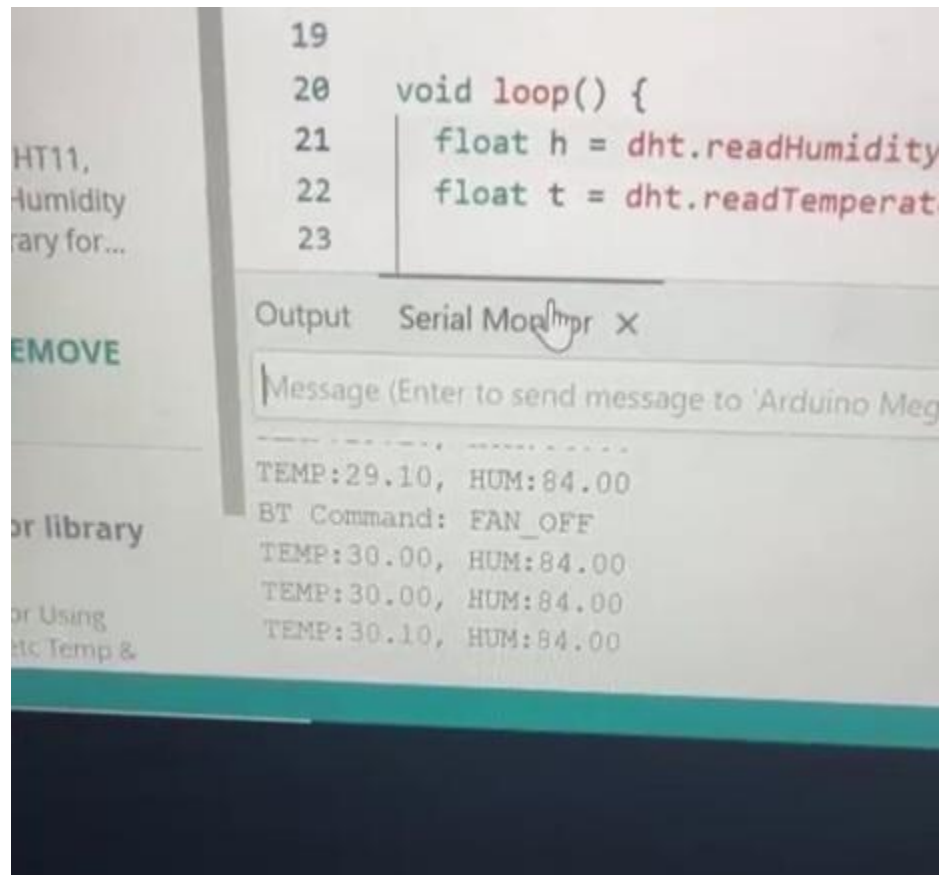
                hum = float(parts[1])
```

```
print(f"Temperature: {temp:.1f} °C | Humidity: {hum:.1f} %")

except:

    print("Parse error")

time.sleep(0.2)
```



DATA COLLECTION

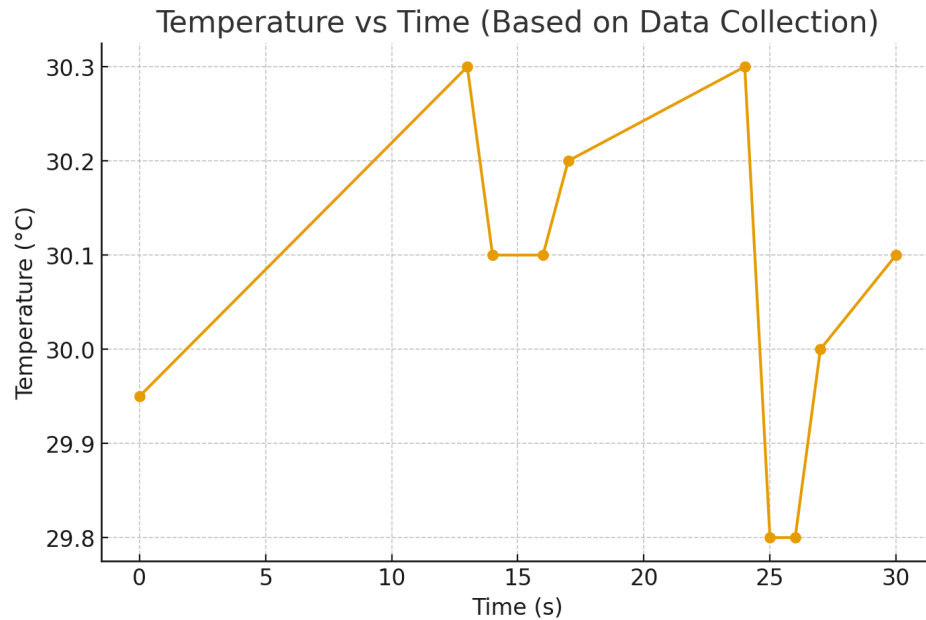
Time range	temperature	Humidity (%)	Command sent	fan/LED state
0.00 - 00:13	29.60-30.30	83-84	No command	FAN OFF(initial state)
00:14-00:16	30.10	84	FAN ON	Turns on
00:17-00:24	30.10-30.30	84	System running with fan on	FAN ON
00:25-00:26	29.80	84	FAN OFF	Turns off
00:27-end	30.0-30.10	84	System running with fan off	FAN OFF

DATA ANALYSIS

The collected data shows how the temperature readings from the DHT22 sensor changed over time and how the system responded to the manual Bluetooth commands “FAN ON” and “FAN OFF.” During the initial period (0:00–0:13), no control commands were sent, and the fan remained OFF. The temperature fluctuated between 29.60°C and 30.30°C, while humidity remained between 83–84%, indicating stable ambient conditions without active cooling. At 00:14–00:16, the “FAN ON” command was sent. The fan (LED) turned ON immediately, confirming correct Bluetooth communication and proper command execution by the Arduino. Following this, between 00:17–00:24, the system continued running with the fan ON, and the temperature stabilized between 30.10°C and 30.30°C. This small variation suggests that either the heat source or the environment did not change significantly, or the fan effect was minimal due to short duration or limited airflow (LED simulation). At 00:25–00:26, the “FAN OFF” command was issued. The fan turned OFF accordingly, demonstrating accurate command processing. After the fan was turned OFF, during 00:27–end, the temperature remained in the range of 30.0°C–30.10°C, indicating the environment stayed relatively stable after fan deactivation.

Overall, the data confirms that the Bluetooth communication using the HC-01 module was functioning correctly, commands were received without delay, and the Arduino responded as programmed. Temperature readings remained stable throughout the experiment, showing no sudden spikes or drops, suggesting consistent sensor performance. Although the temperature changes were minor, the experiment successfully demonstrated the intended closed-loop monitoring and manual control functions.

RESULTS



The graph illustrates the temperature readings collected from the DHT22 sensor over the duration of the experiment. At the start (0–13 seconds), the temperature remains stable around 29.9–30.3°C while the fan/LED is OFF, showing the natural ambient condition without any cooling influence. At approximately 14 seconds, the “FAN ON” command was sent from the smartphone via Bluetooth. This corresponds with the continuation of the temperature curve around 30.1–30.3°C, indicating that the system successfully received the command and switched the fan ON. However, because the fan is simulated using an LED rather than an actual cooling device, no significant temperature drop is observed. The graph stays relatively stable between 30.1–30.3°C, reflecting minimal environmental impact. Around 25 seconds, the “FAN OFF” command was issued. The temperature briefly dips to 29.8°C, likely due to normal sensor fluctuation rather than the effect of cooling, and then continues to stabilize around 30.0–30.1°C. This shows that the system resumed its idle state after receiving the OFF command. Overall, the graph demonstrates that the temperature remained consistent throughout the trial, and the main changes occurred in response to manual Bluetooth commands. The stability of the readings confirms proper functioning of the DHT22 sensor, while the transitions at command points verify the success of the Bluetooth communication and control system.

DISCUSSION

This experiment successfully demonstrated a basic Bluetooth-based temperature monitoring and control system using an Arduino UNO, a DHT22 sensor, and the HC-01 Bluetooth module. The collected data and graph show that the system was able to read temperature values consistently and transmit them wirelessly to a paired smartphone without noticeable delay. The stability of the temperature readings (mostly between 29.8°C and 30.3°C) indicates that the DHT22 sensor was functioning correctly with no major fluctuations or noise in the measurements.

The system also responded accurately to manual control commands sent via the Bluetooth terminal application. When the “FAN ON” command was issued, the Arduino activated the simulated fan (LED), confirming that the HC-01 module received and transferred the command properly. Similarly, the “FAN OFF” command successfully turned the fan off. These transitions demonstrate reliable two-way communication between the smartphone and the Arduino.

Although the fan commands were executed correctly, the temperature graph shows minimal change in temperature during fan ON and OFF periods. This is expected because the experiment used an LED to simulate a fan rather than an actual cooling device. Therefore, the system was able to demonstrate control logic but not real thermal regulation. In a real application with a physical fan or cooling mechanism, a more noticeable drop in temperature would be expected during FAN ON periods.

The overall performance of the system shows that Bluetooth communication is effective for short-range monitoring and control tasks. The experiment also highlights the importance of consistent sampling intervals, stable power supply, and proper pairing procedures to ensure reliable data transmission. Minor delays in sensor readings or slight variations in temperature can be attributed to environmental noise, sensor response time, or Bluetooth transmission latency.

In summary, the system worked as intended: temperature data was measured accurately, transmitted wirelessly, and control commands were received and executed without errors. This validates the practical use of Bluetooth for simple mechatronic control systems and lays the foundation for more advanced applications such as automated environmental control, IoT-based monitoring, or integration with real actuators.

CONCLUSION

This experiment successfully demonstrated a functional Bluetooth-based temperature monitoring and control system using an Arduino UNO, a DHT22 sensor, and an HC-01 Bluetooth module. The Arduino was able to accurately read temperature values and transmit them wirelessly to a smartphone using a Bluetooth terminal application. The system also responded correctly to manual control commands, such as “FAN ON” and “FAN OFF”, by activating or deactivating the simulated fan using the onboard LED.

The temperature data remained stable throughout the test, indicating reliable sensor performance and consistent Bluetooth communication. Although the simulated fan did not cause noticeable temperature changes, the experiment effectively validated the key principles of sensor interfacing, wireless communication, command processing, and basic actuator control. Overall, the system met all experimental objectives and demonstrated the potential of Bluetooth technology for simple remote monitoring and control applications.

RECOMMENDATIONS

1. Use a real fan or actuator : Replacing the LED with an actual small DC fan or relay-controlled device would allow you to observe real temperature changes, leading to more meaningful thermal behavior and control analysis.
2. Implement automatic control mode : Adding an AUTO mode that switches the fan ON/OFF based on a set temperature threshold would demonstrate a complete closed-loop control system.
3. Improve data logging : Instead of viewing temperatures only on a phone, use Python or Excel to automatically save data to a CSV file for more detailed analysis.
4. Increase sampling duration : Running the experiment for a longer period (10–15 minutes) would provide clearer patterns and more stable trend observations.
5. Add noise filtering or averaging : Using a moving average or smoothing technique would reduce minor fluctuations in sensor readings and improve data quality.
6. Enhance user interface : Developing a simple mobile app or GUI could make the system more user-friendly, allowing temperature visualization and command controls in a single interface.

REFERENCES

Aosong Electronics. (2018). DHT22 (AM2302) temperature and humidity sensor datasheet. <https://www.aosong.com/en/products/dht22.html>

Barrett, S. F., & Pack, D. J. (2022). Embedded systems design with the ESP32. Springer.

Handson Technology. (n.d.). HC-05 Bluetooth serial port module datasheet. <https://www.handsontec.com/dataspecs/module/HC-05.pdf>

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>

HowToMechatronics. (n.d.). Arduino and HC-05 Bluetooth module complete tutorial. <https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/>

Monk, S. (2016). Programming Arduino: Getting started with sketches (2nd ed.). McGraw-Hill Education.

MCTA3203 Mechatronics System Integration. (2024). Week 8: Bluetooth data interfacing [Course handout]. Universiti Teknologi MARA.

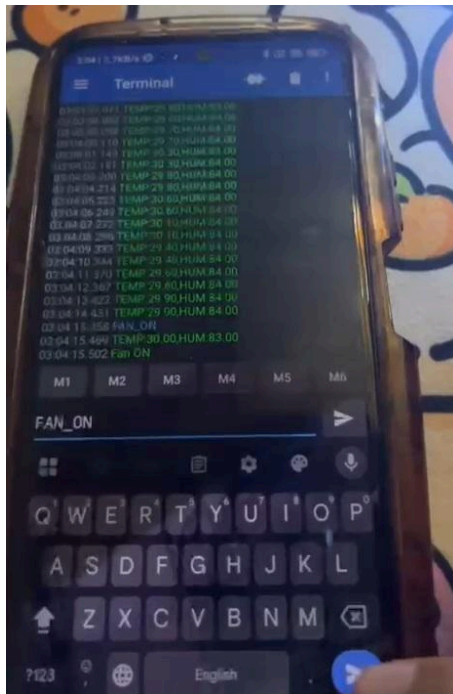
Neil Chaudhary. (n.d.). Arduino Bluetooth basic tutorial. Arduino Project Hub. <https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial-9cff12>

SuperTuris. (n.d.). Basic Bluetooth communication with Arduino & HC-05. Arduino Project Hub.

<https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431c>

Texas Instruments. (2015). Basics of serial communication (Application report). Texas Instruments Inc.

APPENDICES



ACKNOWLEDGEMENTS

Special thanks to Zulkifli Bin Zainal Abidin & Wahju Sediono for their guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been undertaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:

Name: ainin sofiya binti kamaruzaman

Matric Number: 2314916

Signature: *iman*

Name: Muhammad Iman Farihin bin Mohd Imran

Matric Number: 2313813

Signature: 

Name: Muhammad Syazani Akmal Bin Mohd Nasarudin

Matric Number: 23134517